

A fixed field along an unbounded Pell family

Disproof of conjecture 00000000257

ColdSmile — 5 October 2026

The parameter n can grow without bound while $\mathbb{Q}(\sqrt{-(n^2+1)})$ remains the same imaginary quadratic field. Its fixed finite class number therefore cannot satisfy any positive uniform logarithmic lower bound. The entire argument below is formalized in Lean using Mathlib's actual number-field class number.

1 Exact statement and interpretation

The English source reads:

Definition: $h(-d)$ is the class number of the imaginary quadratic field $\mathbb{Q}(\sqrt{-d})$. Conjecture: $h(-(n^2+1)) \geq c \cdot \log n$ holds for an explicit constant $c > 0$ and all $n \geq 2$.

The exact English and Chinese source bytes are supplied in `conjecture.md`. The two versions agree. We use natural-number n (the customary integer interpretation) and the natural logarithm. We prove that *no* positive real constant works, which also rules out an explicit one. The source imposes no squarefree condition and defines a field class number, not a quadratic-order class number.

2 Unbounded negative Pell solutions

For every $k \in \mathbb{N}$ there exist $n, m \in \mathbb{N}$ such that

$$k < n, \quad m > 0, \quad n^2 + 1 = 2m^2. \quad (1)$$

For $k = 0$, use $(n, m) = (1, 1)$. Given a solution for k , set

$$n' = 3n + 4m, \quad m' = 2n + 3m.$$

Then $m' > 0$ and $n' > k + 1$, since $n > k$ and $m \geq 1$. Direct expansion gives

$$(3n + 4m)^2 - 2(2n + 3m)^2 = n^2 - 2m^2 = -1,$$

so (n', m') is a solution for $k + 1$. This proves (1) by induction, including unboundedness; no numerical search or unproved Pell theorem is needed.

3 The field and its class number stay fixed

Let $K_d = \mathbb{Q}(i\sqrt{d}) \subseteq \mathbb{C}$ for $d \in \mathbb{N}$. For $m \in \mathbb{N}$ with $m > 0$,

$$i\sqrt{dm^2} = m i\sqrt{d}, \quad K_{dm^2} = K_d. \quad (2)$$

Both containments follow because multiplication and division by the nonzero rational number m stay inside the field. In particular, every solution in (1) gives $K_{n^2+1} = K_2$. Write $H = h(-2)$, the finite class number of K_2 . Thus

$$h(-(n^2+1)) = H \quad (3)$$

throughout this unbounded family. We do not need to calculate H . The radicand is not being treated as a field discriminant: (2) is an equality of actual embedded fields and hence of their full rings of integers.

4 Contradiction for every positive constant

Fix $c > 0$. Choose $k \in \mathbb{N}$ with $k > \exp(H/c)$. Apply (1) with $k + 2$ to get $n > k + 2$ and $m > 0$. Then $n \geq 2$, (3) holds, and

$$n > \exp(H/c) \implies H/c < \log n \implies h(-(n^2+1)) = H < c \log n.$$

This contradicts the proposed lower bound for that n .

Theorem. *For every $c \in \mathbb{R}$ with $c > 0$, there is $n \in \mathbb{N}$, $n \geq 2$, such that $h(-(n^2+1)) < c \log n$. Consequently conjecture 00000000257 is false.*

5 Faithful formal objects and theorem coverage

All named declarations below belong to namespace `Conjecture257`.

Lean declaration	Mathematical content
<code>imaginaryRoot, quadraticField</code>	$i\sqrt{d}$ and the literal $\mathbb{Q}$ -intermediate field of $\mathbb{C}$ generated by it.
<code>imaginaryRoot_sq,</code> <code>imaginaryRoot_integral</code>	The generator squares to $-d$ and is algebraic over $\mathbb{Q}$.
<code>quadraticFieldNumberField</code>	A genuine characteristic-zero finite extension of $\mathbb{Q}$.
<code>imaginaryRoot_im_pos,</code> <code>definingPolynomial_irreducible,</code> <code>quadraticField_finrank</code>	For $d > 0$, the generator has positive imaginary part and $X^2 + d$ is irreducible over $\mathbb{Q}$; the field has degree exactly two.
<code>imaginaryClassNumber</code>	Mathlib's <code>NumberField.classNumber</code> of this actual field.
<code>quadraticField_mul_square,</code> <code>imaginaryClassNumber_mul_square</code>	Literal equality of embedded fields, then equality of their standard class numbers, for $m \neq 0$.
<code>pell_unbounded</code>	The quantified unboundedness statement (1), proved by induction.
<code>counterexample_for_every_constant</code>	The stronger counterexample theorem for every real $c > 0$.
<code>Conjecture, conjecture_false</code>	The exact positive-constant existence claim and its closed negation.

Mathlib defines the full ring of integers as the integral closure of $\mathbb{Z}$ in the number field. Its class number is the cardinality of the ideal class group, with finiteness proved in the library [1]. This is not an infinite cardinal coerced to zero, an arbitrary function of the radicand, or the class number of a varying nonmaximal order. The intermediate-field model uses the ordinary field generated by the specified root [2]. All field-identity and adequacy statements above are proved in the submitted Lean source; none is assumed as an extra axiom.

The final proposition is exactly

$$\neg \exists c \in \mathbb{R}, \quad 0 < c \wedge \forall n \in \mathbb{N}, \quad 2 \leq n \implies c \log n \leq h(-(n^2 + 1)).$$

A larger domain containing the positive integers cannot rescue the uniform claim. Changing to any fixed logarithm base greater than one only rescales the positive constant, so it does not change the disproof.

6 Verification and reproduction

The complete project uses Lean 4.19.0 and Mathlib v4.19.0 with all nine package revisions pinned. Separate clean project builds and strict replays of all three source modules passed. The audit covers all 17 handwritten declarations (4 definitions, 12 theorems, 1 instance) and all 5 generated logical declarations. All 22 use only `propext`, `Classical.choice`, and `Quot.sound`; their transitive logical dependencies contain no unsafe, partial, or missing declaration. No admitted proof, `native_decide`, or custom axiom is used.

No auxiliary numerical computation supports the mathematics: the unboundedness and field/class-number identities are formal theorems. Verification scripts only execute or inspect the Lean project. Exact source hashes, command logs, complete dependency and axiom inventories, and the independent local semantic review are supplied in `verification/`. These are local validation records; maintainer acceptance remains pending.

7 Commands and provenance

With the pinned Lean toolchain installed, run from the submission folder:

```
cd lean
lake exe cache get
lake build
lake env lean -DwarningAsError=true Conjecture257.lean
lake env lean -DwarningAsError=true Audit257.lean
lake env lean -DwarningAsError=true Check.lean
```

Keep `lake-manifest.json`; it pins Mathlib to commit

c44e0c8ee63ca166450922a373c7409c5d26b00b.

The initial dependency/cache retrieval requires network access. The local verification used fresh project output directories with pinned standard library and Mathlib dependency artifacts; it did not rebuild all of Mathlib from source. The compiler identity and dependency revisions and tracked sources were checked before and after execution.

`Audit257.lean` prints axiom dependencies of every handwritten declaration and instance. `Check.lean` also displays all definitions and theorem types. The separate execution inspector in `verification/` inventories all declarations originating in the submitted modules, including generated ones, and traverses logical dependencies. Its metaprogramming reads existing declarations and supplies no assumption or proof to the mathematical project. `README.md` gives the full inspector command with machine-specific paths supplied explicitly.

The mathematical argument was developed independently from the supplied bilingual statement and pinned Mathlib definitions. No previous competition solution or problem-specific external argument was used. No external class-number computation, approximation to logarithms, or finite numerical prefix is substituted for the quantified theorem. Earlier failed compiler attempts are distinguished from the successful frozen verification records.

The standalone source `report.tex` compiles to the supplied `report.pdf`. The final PDF is visually checked page by page, and its source/output hashes are recorded with the verification evidence.

References

- [1] Mathlib contributors, *Class numbers of number fields*, `Mathlib/NumberTheory/NumberField/ClassNumber.lean`, at the pinned v4.19.0 revision. [Source: class-number definition and proved finiteness.](#)
- [2] Mathlib contributors, *Adjoining elements to intermediate fields*, `Mathlib/FieldTheory/IntermediateField/Adjoin/Defs.lean`, at the same pinned revision. [Source: the generated intermediate field.](#)